

DOCUMENTACIÓN TÉCNICA ACADÉMICA

TechStock

Sistema de Inventario para Componentes Informáticos

Versión: 1.0.0

Fecha: 31 de octubre de 2026

Clasificación: Documentación Técnica Oficial

Arquitectura: MVC · Laravel 12 · SQLite

Laravel 12 | PHP ^8.2 | SQLite 3 | Tailwind CSS v4 | Vite 7

1 Resumen Ejecutivo y Alcance del Sistema

1.1 Descripción General

TechStock es un sistema web de gestión de inventario especializado en el registro, control y trazabilidad de accesorios informáticos. Desarrollado sobre el framework **Laravel 12** con una arquitectura **MVC (Model–View–Controller)**, el sistema provee una interfaz centralizada para la administración integral del ciclo de vida del stock: desde el registro de productos y proveedores, hasta el seguimiento detallado de cada movimiento de inventario mediante un módulo de Kardex.

El sistema está orientado a medianas empresas del sector tecnológico que requieren visibilidad en tiempo real sobre el estado de su inventario, con alertas automáticas ante situaciones de stock crítico y un dashboard ejecutivo con métricas clave del negocio.

1.2 Alcance Funcional

Dominio	Descripción
Autenticación	Gestión de sesiones de usuario con mecanismo seguro de login/logout
Catálogo de Productos	CRUD completo de accesorios informáticos con código SKU único
Categorías	Agrupación taxonómica de productos con protección de integridad referencial
Proveedores	Registro de proveedores con datos de contacto y protección referencial
Kardex / Movimientos	Registro atómico de entradas y salidas de stock con trazabilidad completa
Dashboard Ejecutivo	Panel de métricas en tiempo real: valorización, alertas de stock crítico y distribución por categorías

1.3 Usuarios del Sistema

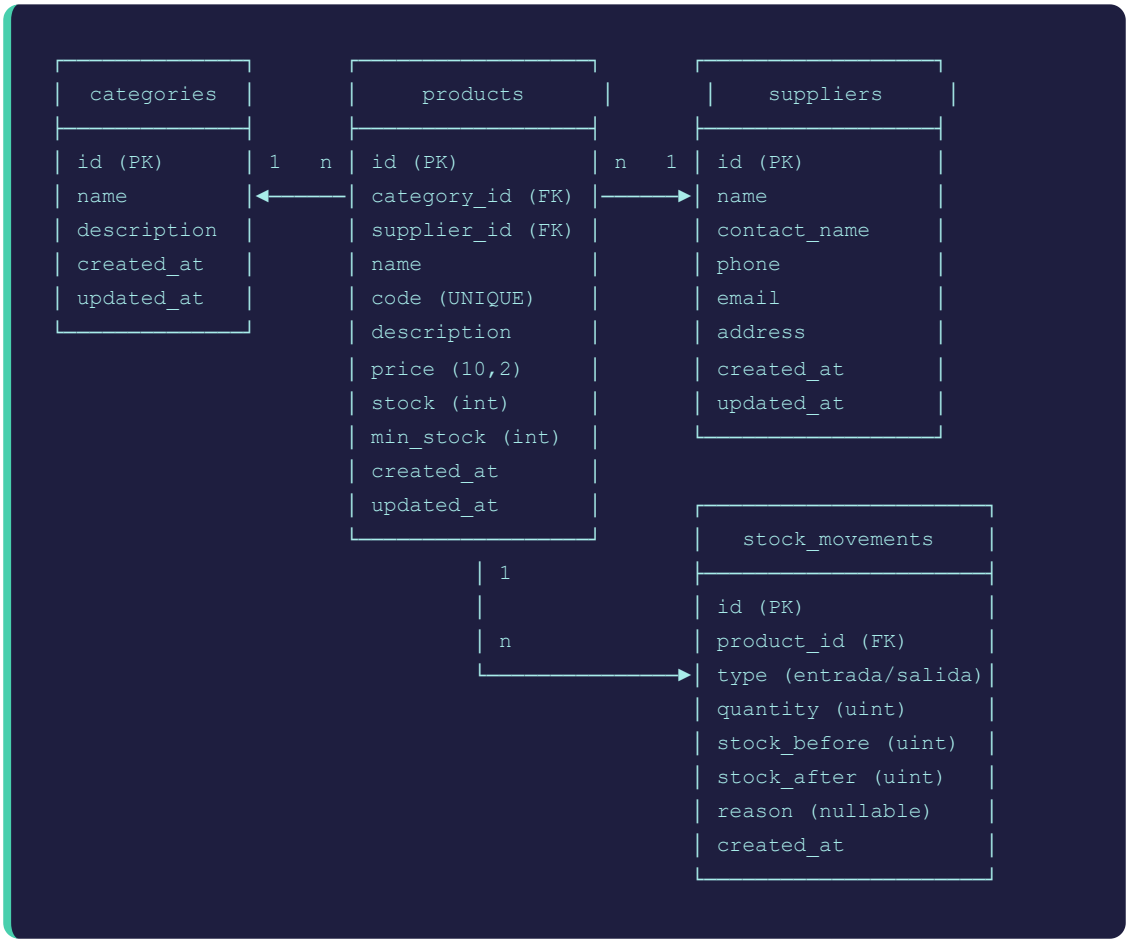
El sistema está concebido para un único rol de usuario autenticado (administrador de inventario), quien dispone de acceso completo a todos los módulos funcionales tras iniciar sesión. Todas las rutas de gestión están protegidas por el middleware de autenticación nativo de Laravel (`auth`).

2 Arquitectura de Base de Datos y Modelos

2.1 Motor de Base de Datos

El sistema utiliza **SQLite** como motor de persistencia, configurado mediante el archivo `database/database.sqlite`. Esta elección responde a la naturaleza del proyecto (entorno académico/de desarrollo), eliminando la necesidad de un servidor de base de datos externo. Laravel gestiona la conexión a través de su capa de abstracción **Eloquent ORM**, lo que garantiza portabilidad hacia motores relacionales más robustos (MySQL, PostgreSQL) sin modificaciones en la lógica de negocio.

2.2 Esquema Relacional — Diagrama de Entidades



2.3 Descripción de Entidades

2.3.1 Entidad `categories` (Categorías)

Columna	Tipo	Restricciones	Descripción
`id`	INTEGER	PK, AUTO_INCREMENT	Identificador único
`name`	VARCHAR(100)	NOT NULL, UNIQUE	Nombre descriptivo de la categoría
`description`	TEXT	NULLABLE	Descripción extendida
`created_at`	TIMESTAMP	—	Fecha de creación (automático)
`updated_at`	TIMESTAMP	—	Fecha de modificación (automático)

Relaciones: Una categoría puede contener múltiples productos (`HasMany → Product`). La eliminación está restringida si existen productos asociados (`restrictOnDelete`).

2.3.2 Entidad `suppliers` (Proveedores)

Columna	Tipo	Restricciones	Descripción
`id`	INTEGER	PK, AUTO_INCREMENT	Identificador único
`name`	VARCHAR(150)	NOT NULL	Razón social o nombre comercial
`contact_name`	VARCHAR(100)	NULLABLE	Nombre del contacto principal
`phone`	VARCHAR(20)	NULLABLE	Número de teléfono
`email`	VARCHAR(100)	NULLABLE	Correo electrónico de contacto
`address`	VARCHAR(255)	NULLABLE	Dirección física del proveedor
`created_at`	TIMESTAMP	—	Fecha de creación (automático)
`updated_at`	TIMESTAMP	—	Fecha de modificación (automático)

2.3.3 Entidad `products` (Productos)

Entidad central del sistema. Almacena el catálogo completo de accesorios informáticos.

Columna	Tipo	Restricciones	Descripción
`id`	INTEGER	PK, AUTO_INCREMENT	Identificador único
`name`	VARCHAR(150)	NOT NULL	Nombre descriptivo del accesorio
`code`	VARCHAR(50)	NOT NULL, UNIQUE	Código SKU único del producto
`description`	TEXT	NULLABLE	Descripción técnica detallada
`price`	DECIMAL(10,2)	NOT NULL	Precio unitario del producto
`stock`	INTEGER	DEFAULT 0	Cantidad actual en inventario
`min_stock`	INTEGER	DEFAULT 5	Umbral mínimo de alerta de stock crítico
`category_id`	INTEGER	FK → categories.id	Categoría a la que pertenece
`supplier_id`	INTEGER	FK → suppliers.id	Proveedor que lo suministra
`created_at`	TIMESTAMP	—	Fecha de registro (automático)
`updated_at`	TIMESTAMP	—	Fecha de modificación (automático)

2.3.4 Entidad `stock_movements` (Kardex)

Columna	Tipo	Restricciones	Descripción
`id`	INTEGER	PK, AUTO_INCREMENT	Identificador único del movimiento
`product_id`	INTEGER	FK → products.id	Producto al que corresponde el movimiento
`type`	VARCHAR	in: entrada, salida	Tipo de movimiento de inventario

`quantity`	UNSIGNED INT	NOT NULL	Cantidad de unidades movidas
`stock_before`	UNSIGNED INT	NOT NULL	Stock antes del movimiento
`stock_after`	UNSIGNED INT	NOT NULL	Stock después del movimiento
`reason`	VARCHAR(255)	NULLABLE	Motivo o descripción del movimiento
`created_at`	TIMESTAMP	—	Fecha y hora exacta del movimiento

2.4 Integridad Referencial

El esquema implementa **integridad referencial estricta** mediante la directiva `restrictOnDelete()` en todas las claves foráneas:

- No es posible eliminar una **categoría** si tiene productos asociados.
- No es posible eliminar un **proveedor** si tiene productos asociados.
- No es posible eliminar un **producto** si posee movimientos de stock en el Kardex.

3 Descripción Funcional de los Módulos

3.1 Módulo de Autenticación

Controlador: `AuthController` · **Rutas:** `GET /login`, `POST /login`, `POST /logout`

El módulo utiliza el sistema de autenticación nativo de Laravel (`IlluminateSupportFacadesAuth`). Sus responsabilidades son:

- **`showLoginForm()`**: Renderiza el formulario de inicio de sesión. Si el usuario ya tiene sesión activa, redirige automáticamente al Dashboard.
- **`login()`**: Valida las credenciales, regenera el token de sesión para prevenir ataques de fijación de sesión, y soporta la opción *"Recordarme"* mediante cookie persistente.
- **`logout()`**: Invalida la sesión activa, regenera el token CSRF y redirige al login.

3.2 Módulo Dashboard (Panel de Control Ejecutivo)

Controlador: `DashboardController` · **Ruta:** `GET /dashboard`

El Dashboard es el módulo central de inteligencia operacional del sistema, computando las siguientes métricas en tiempo real:

Métrica	Implementación	Descripción
Total de Productos	`Product::count()`	Número total de accesorios en el catálogo
Total de Categorías	`Category::count()`	Número de categorías activas
Total de Proveedores	`Supplier::count()`	Número de proveedores registrados
Stock Crítico	`whereColumn('stock', '<=', 'min_stock')`	Productos bajo su umbral mínimo

3.2.1 Valorización Total del Inventario

```
$inventoryValue = Product::selectRaw('SUM(price * stock) as total_value')
    ->value('total_value') ?? 0;
```

Calcula mediante SQL agregado el valor total del inventario multiplicando `price × stock` por cada producto. Esta métrica es fundamental para la toma de decisiones financieras.

3.2.2 Control de Stock Crítico

```
$lowStockProducts = Product::with(['category', 'supplier'])
    ->whereColumn('stock', '<=', 'min_stock')
    ->orderBy('stock')
    ->get();
```

Identifica dinámicamente todos los productos cuyo stock actual ≤ umbral mínimo (`min_stock`), ordenados de menor a mayor para priorizar los casos más urgentes. Utiliza eager loading para evitar el problema N+1.

3.2.3 Distribución por Categorías y Movimientos Recientes

```
$categoryDistribution = Category::withCount('products')->orderByDesc('produ
$recentMovements      = StockMovement::with('product')->latest()->take(6)->
```

3.3 Módulo de Categorías

Controlador: `CategoryController` · **Rutas:** Resource `/categories` (excluye `show`)

- Listado paginado con conteo de productos asociados (`withCount('products')`) a 10 registros por página.
- Validación de unicidad del nombre con `Rule::unique()->ignore($id)` en edición.
- Protección de eliminación: verifica productos dependientes antes de ejecutar el `DELETE`.

3.4 Módulo de Proveedores

Controlador: `SupplierController` · **Rutas:** Resource `/suppliers` (excluye `show`)

- Listado paginado con conteo de productos vinculados.
- Validación de correo electrónico con la regla `email` nativa de Laravel.
- Campos opcionales: `contact_name`, `phone`, `email`, `address`.
- Protección de eliminación idéntica al módulo de categorías.

3.5 Módulo de Productos

Controlador: `ProductController` · **Rutas:** Resource `/products` (excluye `show`)

3.5.1 Sistema de Filtros Combinados

Parámetro	Tipo	Implementación SQL
`search`	Texto libre	`WHERE name LIKE %x% OR code LIKE %x%`
`category_id`	Filtro exacto	`WHERE category_id = x`
`supplier_id`	Filtro exacto	`WHERE supplier_id = x`
`stock_status`	Condicional	`stock <= min_stock` / `stock > min_stock`

3.5.2 Protección de Integridad en Eliminación

```
public function destroy(Product $product)
{
    if ($product->stockMovements()->exists()) {
        return redirect()->route('products.index')
            ->with('error', 'No se puede eliminar el producto porque
                             registra movimientos de stock en el Kardex.');
    }
    $product->delete();
}
```

3.6 Módulo de Movimientos de Stock (Kardex)

Controlador: `StockMovementController` · **Rutas:** `GET /stock-movements`, `POST /stock-movements`

Este módulo implementa la trazabilidad completa del inventario mediante **transacciones atómicas de base de datos**:

```
DB::transaction(function () use ($request) {
    $product = Product::where('id', $request->product_id)
        ->lockForUpdate()   // Bloqueo de fila para concurrencia segura
        ->firstOrFail();

    $stockBefore = (int) $product->stock;
    $quantity    = (int) $request->quantity;

    if ($request->type === 'salida') {
        if ($quantity > $stockBefore) {
            throw ValidationException::withMessages([...]);
        }
        $stockAfter = $stockBefore - $quantity;
    } else {
        $stockAfter = $stockBefore + $quantity;
    }
```

```
    StockMovement::create([...]);          // Registro inmutable de trazabi
    $product->update(['stock' => $stockAfter]); // Actualización del stock
});
```

Característica	Descripción
Atomicidad	El registro del movimiento y la actualización del stock son una operación indivisible (ROLLBACK automático ante fallos).
Bloqueo de fila (`lockForUpdate`)	Previene condiciones de carrera en acceso concurrente.
Validación de stock	Para salidas, verifica que la cantidad no supere el stock disponible.
Trazabilidad completa	Se registran `stock_before` y `stock_after` en cada movimiento.

Tipo	Efecto	Descripción
`entrada`	`stock_after = stock_before + quantity`	Recepción, devoluciones, ajuste positivo
`salida`	`stock_after = stock_before - quantity`	Venta, despacho, pérdida, ajuste negativo

4 Especificaciones Técnicas del Stack

4.1 Backend

Componente	Tecnología	Versión	Función
Lenguaje	PHP	^8.2	Lenguaje de programación del servidor
Framework	Laravel	^12.0	Framework MVC principal
ORM	Eloquent ORM	(integrado)	Mapeo objeto-relacional y gestión de consultas
Motor DB	SQLite	3.x	Sistema de gestión de base de datos embebido
CLI	Artisan	(integrado)	Migraciones, seeders y gestión del proyecto
REPL	Laravel Tinker	^2.10.1	Consola interactiva para depuración

4.2 Frontend

Componente	Tecnología	Versión	Función
Plantillas	Blade (Laravel)	(integrado)	Motor de templating del servidor
Framework CSS	Tailwind CSS	^4.0.0	Framework CSS de utilidad, responsivo
Plugin Tailwind	@tailwindcss/vite	^4.0.0	Integración Tailwind 4 con Vite
Bundler	Vite	^7.0.7	Construcción y servidor de desarrollo HMR
Plugin Vite	laravel-vite-plugin	^2.0.0	Integración oficial Vite ↔ Laravel
HTTP Client	Axios	^1.11.0	Librería para peticiones HTTP asíncronas

4.3 Herramientas de Desarrollo

Herramienta	Versión	Propósito
Laravel Pint	^1.24	Formateador de código PHP (PSR-12)
Laravel Sail	^1.41	Entorno de desarrollo Docker
Laravel Pail	^1.2.2	Visualizador de logs en tiempo real
PHPUnit	^11.5.50	Framework de pruebas unitarias
Faker PHP	^1.23	Generación de datos ficticios para seeders
Mockery	^1.6	Librería de mocking para pruebas
Concurrently	^9.0.1	Ejecución simultánea de múltiples procesos

4.4 Enrutamiento RESTful

Método	URI	Controlador@Método	Nombre de Ruta
GET	/login	AuthController@showLoginForm	login
POST	/login	AuthController@login	login.submit
POST	/logout	AuthController@logout	logout
GET	/dashboard	DashboardController@index	dashboard
GET	/categories	CategoryController@index	categories.index
POST	/categories	CategoryController@store	categories.store
GET	/categories/{id}/edit	CategoryController@edit	categories.edit
PUT/PATCH	/categories/{id}	CategoryController@update	categories.update
DELETE	/categories/{id}	CategoryController@destroy	categories.destroy
GET	/suppliers, /products	(análogo a categories)	suppliers.*, products.*
GET	/stock-movements	StockMovementController@index	stock-movements.index
POST	/stock-movements	StockMovementController@store	stock-movements.store

4.5 Patrones de Software Aplicados

Patrón	Aplicación en el Proyecto
MVC	Separación estricta entre Modelos (Eloquent), Vistas (Blade) y Controladores
Unit of Work	`DB::transaction()` en StockMovementController garantiza consistencia multi-paso
Active Record	Los modelos Eloquent encapsulan datos y operaciones de persistencia
Route Model Binding	Resolución automática de instancias de modelo desde parámetros de ruta
Eager Loading	Uso de `with(['relation'])` para prevenir el problema N+1

> 📌 **Nota de auditoría:** Los movimientos de stock (Kardex) son inmutables por diseño — no pueden ser editados ni eliminados desde la interfaz, garantizando un historial de auditoría inviolable para todos los cambios de inventario.

Informe Técnico TechStock v1.0.0 — Generado el 31 de octubre de 2026
Documentación basada en el análisis del código fuente real del proyecto.